DIRECCIÓN DE EDUCACIÓN A DISTANCIA
Diseño y Programación de Software Multiplataforma DPS941 (Virtual)

DOCENTE:
Alexander Alberto Siguenza Campos

ACTIVIDAD:
Foro 2
Firebase Authentication

PROYECTO:
Sonrisa Digital
Sistema multiplataforma de gestión para la Clínica Dental Sonrisa Perfecta

EQUIPO:
404 DPS Devs

INTEGRANTES:

Carné	Nombres completos
MV252134	Milton Antonio Mayorga Vásquez
CP121738	David Guillermo Cardona Pérez
CM210922	Rene Gerardo Castillo Monterrosa
OA182314	Mario Alejandro Orellana Andrade
RS221448	Juan Diego Rodríguez Somoza

URL DEL REPOSITORIO
https://github.com/MiltonMV2025/DPS_PROYECTO

VIDEO DE DEMOSTRACIÓN
https://www.youtube.com/watch?v=9oOkpD04vwM

FECHA DE ENTREGA
8 de noviembre de 2026

Contenido

II. Objetivos 4

2.1. Objetivo general 4

2.2. Objetivos específicos 4

III. Firebase y su utilidad en React Native 5

3.1. Elección de React Native Firebase 5

3.2. Compilación de desarrollo 6

IV. Configuración de Firebase en el proyecto 6

4.1. Proyecto de Firebase y proveedor de Google 6

4.2. Archivos de configuración y credenciales 7

4.3. Cambios en la base de datos 8

V. Registro e inicio de sesión con Google 9

5.1. Inicio de sesión 9

5.2. Registro de pacientes nuevos 11

5.3. Inicio de sesión con correo y contraseña 13

5.4. Mensajes de error 13

5.5. Sesión y cierre de sesión 13

VI. Datos del paciente y su relación con la cuenta 14

VII. Seguridad de la información 15

VIII. Módulos secundarios e historial de registros 16

IX. Notificaciones con Firebase Cloud Messaging 17

9.1. Avance actual 17

9.2. Separación de responsabilidades 18

9.3. Qué token obtiene la aplicación 18

9.4. Actividades pendientes 19

X. Pruebas 20

XI. Análisis y limitaciones 22

XII. Conclusiones 23

XIII. Referencias 24

I. Introducción

Sonrisa Digital es un sistema multiplataforma desarrollado para apoyar la gestión de la Clínica Dental Sonrisa Perfecta. El panel web facilita el trabajo administrativo y clínico, mientras que la aplicación móvil permite al paciente iniciar sesión, consultar sus citas, solicitar nuevos horarios, revisar su historial clínico y actualizar algunos datos personales. Ambas plataformas utilizan una misma API y comparten la información almacenada en MySQL (404 DPS Devs, 2026a).

En la etapa anterior se documentó el desarrollo móvil con React Native, Expo y el diseño de interfaces en Figma. En esta entrega se incorpora Firebase a la aplicación, principalmente a través de Firebase Authentication, que ahora permite a los pacientes registrarse e iniciar sesión con su cuenta de Google. También se retoma Firebase Cloud Messaging (FCM) como medio para enviar recordatorios de citas, una necesidad que la clínica planteó desde el inicio del proyecto.

Este informe distingue entre las funciones que ya están implementadas y las actividades que aún se deben realizar. No se presentan pruebas ni resultados que todavía no han sido obtenidos.

II. Objetivos

2.1. Objetivo general

Integrar Firebase en la aplicación móvil Sonrisa Digital, desarrollada en React Native y Expo, para que los pacientes puedan autenticarse con su cuenta de Google, manteniendo la seguridad de la información y la arquitectura compartida con el sistema web.

2.2. Objetivos específicos

- Configurar Firebase Authentication con el proveedor de Google y preparar la compilación de desarrollo que necesita la aplicación.
- Describir el flujo de registro e inicio de sesión con Google, desde el teléfono del paciente hasta la validación en el servidor.
- Establecer cómo se asocian los registros del paciente a su cuenta y qué medidas protegen esa información.
- Definir el procedimiento para enviar recordatorios de citas mediante Firebase Cloud Messaging.

III. Firebase y su utilidad en React Native

Firebase es una plataforma de Google que reúne servicios para aplicaciones, entre ellos autenticación, almacenamiento, bases de datos y mensajería. React Native puede conectarse a algunos de estos servicios mediante el SDK de JavaScript de Firebase o mediante bibliotecas nativas. Expo distingue ambas alternativas: el SDK web es útil para Auth, Firestore o Storage, mientras que los módulos nativos requieren una compilación de desarrollo y no funcionan como simples extensiones dentro de Expo Go (Expo, s. f.-a).

De todos estos servicios, el que más aporta al proyecto en este momento es Authentication. Permite identificar a una persona por medio de proveedores externos, como Google, y entrega a la aplicación un token de identificación que el servidor puede verificar (Firebase, s. f.-a). Para la clínica esto significa que un paciente puede crear su cuenta con el correo que ya utiliza en su teléfono, sin tener que recordar una contraseña más.

3.1. Elección de React Native Firebase

Para conectar la aplicación con Firebase se evaluaron las dos alternativas. El SDK de JavaScript tenía la ventaja de seguir funcionando en Expo Go, pero con él el inicio de sesión con Google necesita abrir un flujo web para obtener el token. React Native Firebase, en cambio, envuelve los SDK nativos de Android e iOS (Invertase, s. f.-a) y permite usar directamente el selector de cuentas del teléfono, que es lo que el paciente ya conoce de otras aplicaciones. Por esa razón se eligió esta segunda opción. La Tabla 1 resume las diferencias que se tomaron en cuenta.

Tabla 1

Comparación entre las alternativas para integrar Firebase en React Native

Criterio	SDK de JavaScript	React Native Firebase
Funciona en Expo Go	Sí	No; requiere una compilación de desarrollo
Inicio de sesión con Google	Necesita un flujo web para obtener el token	Usa el selector de cuentas del teléfono

Criterio	SDK de JavaScript	React Native Firebase
Configuración	Objeto de configuración dentro del código	Archivos `google-services.json` y `GoogleService-Info.plist`
Opción elegida	No	Sí

Nota. Elaboración propia a partir de Expo (s. f.-a) e Invertase (s. f.-a).

En el proyecto se instalaron los paquetes `@react-native-firebase/app` y `@react-native-firebase/auth`, junto con `react-native-nitro-google-signin`, que se encarga de mostrar el selector de cuentas y de devolver el token de Google.

3.2. Compilación de desarrollo

La consecuencia más visible de esta elección es que el inicio de sesión ya no puede probarse en Expo Go. Expo facilita mucho la creación y el arranque de un proyecto de React Native (Román Gallardo et al., 2020), pero cuando una biblioteca incluye código nativo hace falta una compilación de desarrollo, es decir, una versión de la aplicación que trae esas bibliotecas y la herramienta `expo-dev-client` (Expo, s. f.-b).

Desde ahora la aplicación se instala en el emulador con el comando `npx expo run:android`. Expo Go solo se usa para revisar pantallas que no dependen de módulos nativos y, si alguien intenta entrar con Google desde ahí, la aplicación muestra un aviso que indica que se necesita la compilación de desarrollo.

IV. Configuración de Firebase en el proyecto

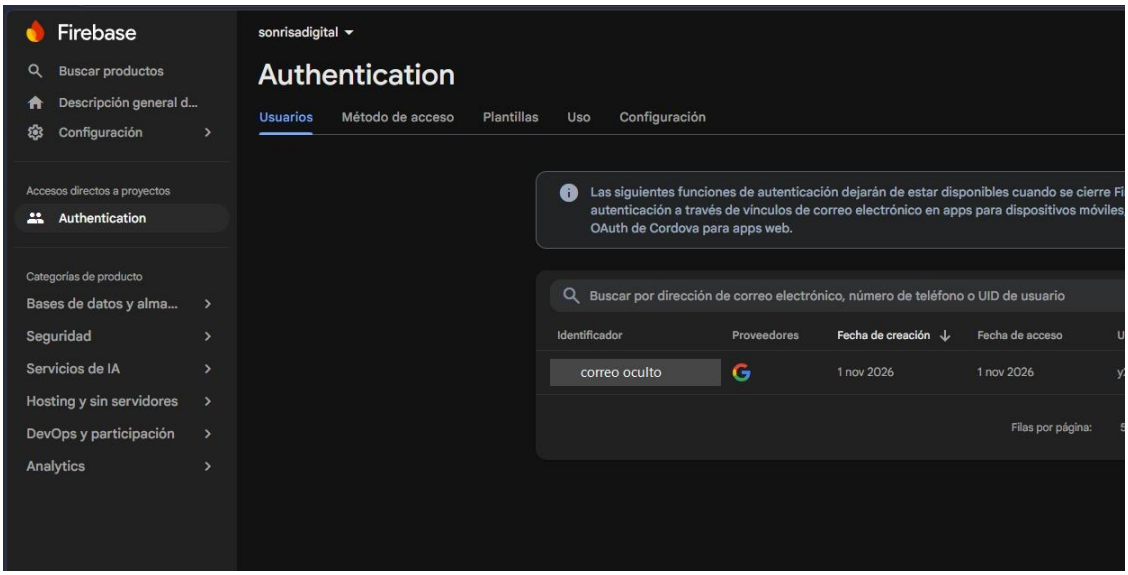
4.1. Proyecto de Firebase y proveedor de Google

En la consola de Firebase se creó el proyecto sonrisadigital y, dentro de Authentication, se habilitó Google como método de acceso. Después se registró la aplicación Android con el identificador `com.sonrisadigital.mobile`, el mismo que aparece en `app.json`, y se agregó la huella SHA-1 del certificado de desarrollo. Este último paso es fácil de pasar por alto, pero sin la huella Google no puede confirmar que la solicitud proviene de la

aplicación registrada y rechaza el inicio de sesión (Invertase, s. f.-b). En la Figura 1 se observa la consola con un usuario que ya inició sesión con Google.

Figura 1

Consola de Firebase Authentication con un usuario registrado mediante Google



Nota. Captura de la consola del proyecto sonrisadigital. Se ocultó el correo electrónico del usuario para proteger sus datos personales.

4.2. Archivos de configuración y credenciales

Firebase identifica a la aplicación por medio del archivo `google-services.json` en Android y `GoogleService-Info.plist` en iOS; en `app.json` se indica dónde se encuentra cada uno (Expo, s. f.-e). Como estos archivos contienen datos del proyecto, se agregaron al `.gitignore` y cada integrante los descarga desde la consola cuando necesita compilar la aplicación.

El servidor, por su parte, utiliza el SDK de Firebase Admin para verificar los tokens, y para ello necesita las credenciales de una cuenta de servicio (Firebase, s. f.-c). Esas credenciales se guardan solo como variables de entorno del backend, tanto en el entorno local como en Vercel. Un cuidado importante fue no ponerles nunca el prefijo `EXPO_PUBLIC_`, ya que todo lo que lleva ese prefijo se incluye dentro de la aplicación y cualquier persona podría

extraerlo. De esta manera, el repositorio público no contiene ninguna clave. La Tabla 2 muestra dónde se encuentra cada elemento de la configuración.

Tabla 2
Elementos de configuración de Firebase y su ubicación

Elemento	Ubicación	¿Se sube a Git?
`google-services.json`	Carpeta `mobile/` (Android)	No
`GoogleService-Info.plist`	Carpeta `mobile/` (iOS)	No
Complementos de Firebase y Google	`mobile/app.json`	Sí
Credenciales de Firebase Admin	Variables de entorno del servidor (local y Vercel)	No
Columna `firebase_uid`	Migración `007_firebase_auth.sql`	Sí
Cuentas sin contraseña local	Migración `008_google_patient_registration.sql`	Sí

Nota. Elaboración propia a partir del repositorio del proyecto (404 DPS Devs, 2026a).

4.3. Cambios en la base de datos

Para relacionar las cuentas de Firebase con los usuarios que ya existían en la clínica se agregaron dos migraciones. La primera, `007_firebase_auth.sql`, añade a la tabla Usuarios la columna `firebase_uid`, donde se guarda el identificador que Firebase asigna a cada cuenta. La columna acepta valores nulos, porque los usuarios existentes no tienen ese dato hasta que entran con Google por primera vez, y es única, para que una misma cuenta de Google no quede asociada a dos personas.

La segunda, `008_google_patient_registration.sql`, permite que la contraseña quede vacía en el caso de los pacientes que se registran solo con Google. Esas cuentas no pueden usarse en el formulario de correo y contraseña, y las contraseñas que ya existían no se modificaron.

V. Registro e inicio de sesión con Google

5.1. Inicio de sesión

Cuando el paciente pulsa «Continuar con Google» intervienen tres partes: el selector de cuentas de Google, Firebase Authentication y la API de Sonrisa Digital. La Tabla 3 describe cada paso y la Figura 2 muestra el mismo recorrido como un diagrama de secuencia.

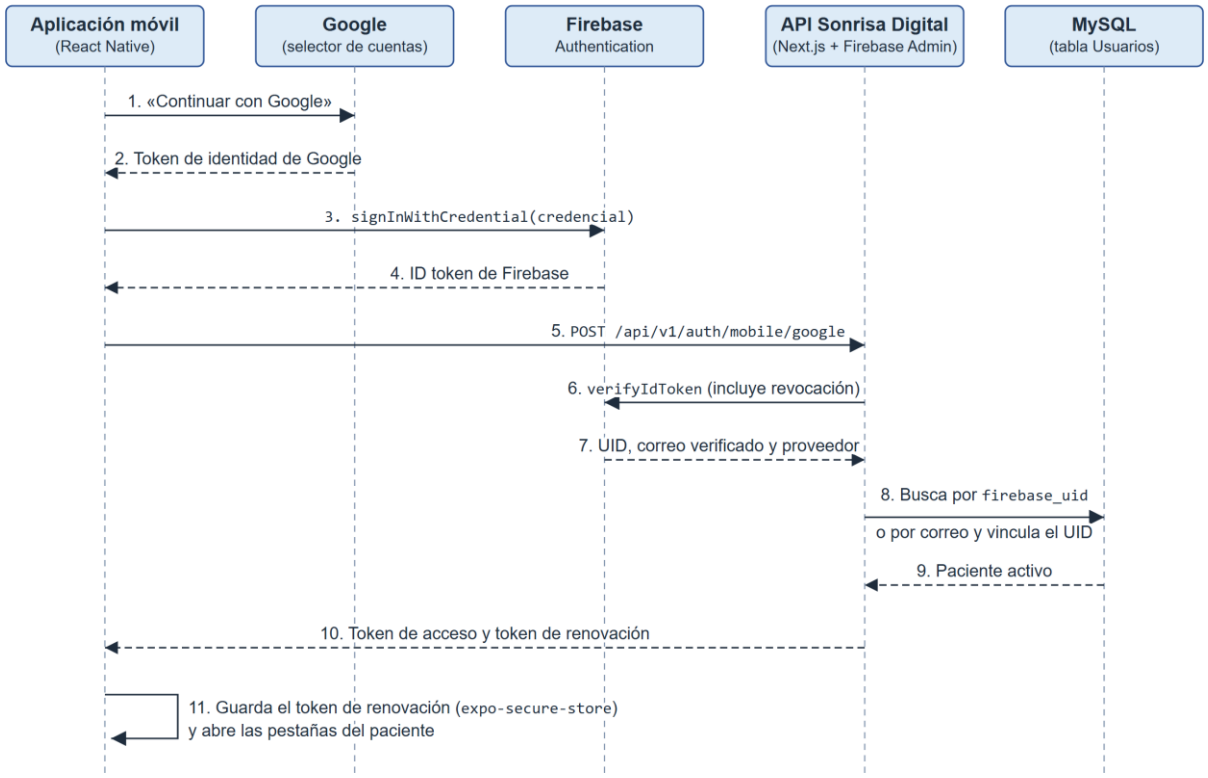
Tabla 3
Pasos del inicio de sesión con Google

Paso	Componente	Acción
1	Aplicación (pantalla de acceso)	El paciente pulsa «Continuar con Google».
2	Selector de cuentas de Google	Muestra las cuentas del teléfono y devuelve un token de identidad de Google.
3	Firebase Authentication	Convierte ese token en una credencial e inicia sesión con `signInWithCredential`; Firebase emite su propio token de identificación.
4	Aplicación → API	Envía el token de Firebase a `POST /api/v1/auth/mobile/google`.
5	API (Firebase Admin)	Verifica el token con `verifyIdToken`, incluida la revisión de tokens revocados, y exige que el correo esté verificado y que el proveedor sea Google.
6	API (servicio de identidad)	Busca al usuario por `firebase_uid`; si no lo encuentra, lo busca por correo y guarda el identificador. Solo acepta pacientes con cuenta activa.
7	API → aplicación	Emite la sesión del sistema: un token de acceso de corta duración y un token de renovación.
8	Aplicación	Guarda el token de renovación con `expo-secure-store` y muestra las pestañas del paciente.

Nota. Elaboración propia a partir del código de la aplicación móvil y del backend (404 DPS Devs, 2026a).

Figura 2

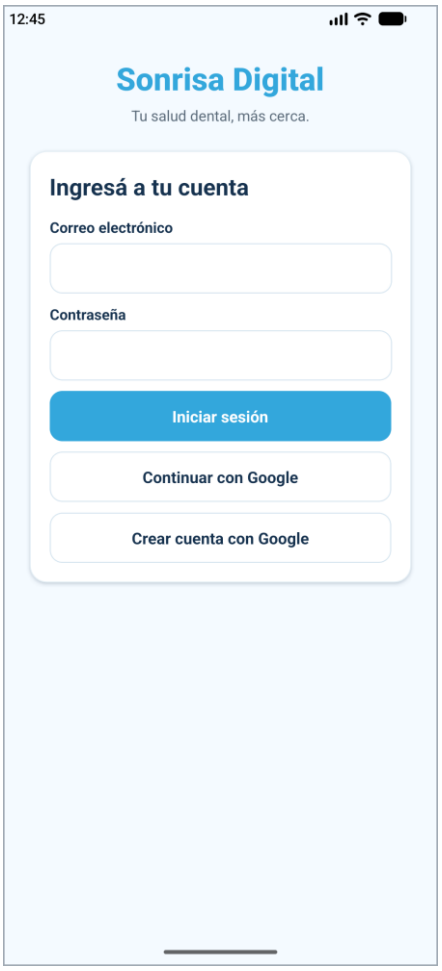
Diagrama de secuencia del inicio de sesión con Google



Nota. Elaboración propia. Las flechas continuas representan solicitudes y las discontinuas, respuestas.

Un punto que conviene aclarar es por qué la API emite su propia sesión en lugar de usar directamente el token de Firebase. Firebase sirve para confirmar quién es la persona, pero los roles y permisos de Sonrisa Digital están en MySQL, y los demás servicios (citas, historial y perfil) ya trabajaban con los tokens del sistema. Por eso, una vez verificado el token de Firebase (Firebase, s. f.-b), la API entrega la misma sesión que entregaría con correo y contraseña, y el resto de la aplicación no necesitó cambios.

Figura 3
Pantalla de acceso de la aplicación móvil



Nota. Elaboración propia.

5.2. Registro de pacientes nuevos

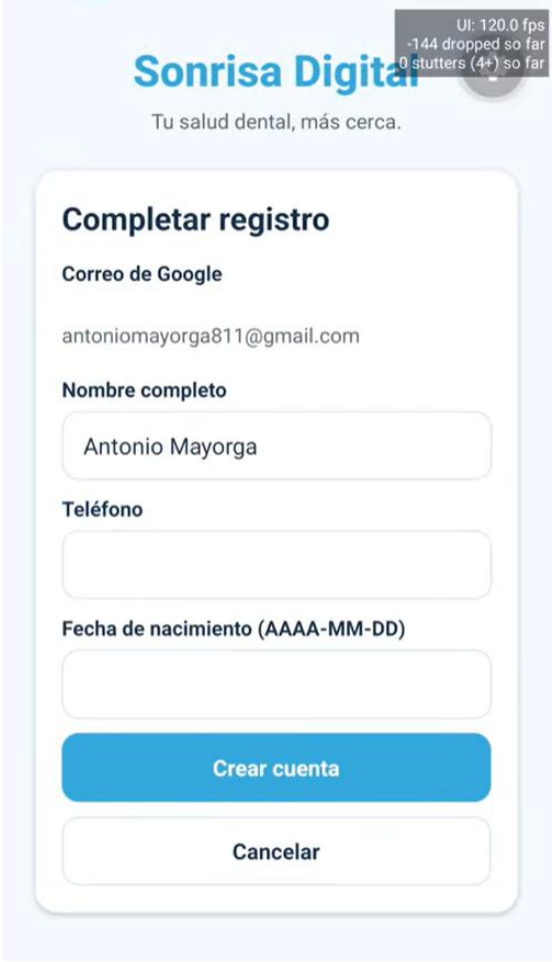
Para los pacientes que todavía no tienen cuenta se agregó el botón «Crear cuenta con Google». Después de elegir la cuenta, la aplicación muestra el formulario «Completar registro». El correo de Google aparece como dato fijo y el nombre completo viene precargado desde la cuenta, aunque el paciente puede corregirlo; además, se solicitan el teléfono y la fecha de nacimiento (en formato AAAA-MM-DD), que la clínica necesita para abrir el expediente. Al pulsar «Crear cuenta», la aplicación envía estos datos junto con el token de Firebase a `POST /api/v1/auth/mobile/google/register`.

Antes de crear la cuenta, el servidor vuelve a verificar el token de Firebase y valida los datos con esquemas de Zod: el nombre debe tener entre 3 y 120 caracteres, el teléfono

entre 7 y 20 dígitos o signos permitidos, y la fecha de nacimiento tiene que ser anterior a la fecha actual. Si el correo o la cuenta de Google ya pertenecen a otro usuario, el registro se rechaza. El usuario y su perfil de paciente se guardan en una misma transacción, de modo que, si algo falla a la mitad, no queda un registro incompleto.

Desde la aplicación no es posible elegir un rol: toda cuenta creada de esta forma es de paciente. Si la persona pulsa «Cancelar», la aplicación cierra la sesión de Firebase y de Google para no dejarla abierta a medias. Cuando el registro termina correctamente, el paciente entra directamente a la aplicación con su sesión iniciada.

Figura 4
Formulario de registro con Google



Nota. Elaboración propia.

5.3. Inicio de sesión con correo y contraseña

El inicio de sesión con correo y contraseña sigue funcionando como en la entrega anterior, a través de la API del proyecto, que compara la contraseña ingresada con su hash bcrypt. Como entrega el mismo tipo de sesión que el acceso con Google, el resto de la aplicación trata igual a ambos usuarios.

5.4. Mensajes de error

Para que el paciente entienda qué ocurrió cuando no puede entrar, se definió una respuesta para cada situación. La Tabla 4 recoge los casos contemplados.

Tabla 4
Situaciones de error en el inicio de sesión con Google

Situación	Respuesta de la aplicación
El paciente cierra el selector de cuentas	Vuelve a la pantalla de acceso sin mostrar error.
El correo de Google no pertenece a una cuenta de la clínica	«No encontramos una cuenta clínica activa con este correo.»
La cuenta está desactivada	«La cuenta está desactivada.»
El usuario no tiene el rol de paciente	«La aplicación Mobile está disponible para cuentas de paciente.»
La cuenta de Google ya está vinculada a otro usuario	«Esta cuenta de Google ya está asociada a otro usuario.»
El token de Firebase no es válido, venció o fue revocado	«No se pudo validar la cuenta de Google.»
La aplicación se abrió desde Expo Go	«Google Sign-In requiere una development build de la aplicación.»

Nota. Elaboración propia a partir de los mensajes definidos en el código (404 DPS Devs, 2026a).

5.5. Sesión y cierre de sesión

El token de acceso se mantiene solo en memoria y el de renovación se guarda con `expo-secure-store`, que cifra la información con el Keychain en iOS y el Keystore en Android (Expo, s. f.-d). Cada vez que la sesión se renueva, el servidor entrega un token de renovación nuevo y guarda únicamente su hash; si detecta que alguien intenta usar uno que ya fue reemplazado, revoca la sesión completa.

Al cerrar sesión, la aplicación revoca la sesión en el servidor, desregistra el dispositivo de las notificaciones, borra la caché y el almacenamiento seguro, y cierra la sesión de Firebase y de Google. Así se evita que, si otra persona utiliza el mismo teléfono, vea los datos del paciente anterior o entre con su cuenta sin tener que elegirla.

VI. Datos del paciente y su relación con la cuenta

Firebase solo guarda la cuenta de autenticación, es decir, el identificador, el correo y el proveedor. Las citas, los historiales y el resto de la información clínica permanecen en MySQL, que es la base de datos del sistema.

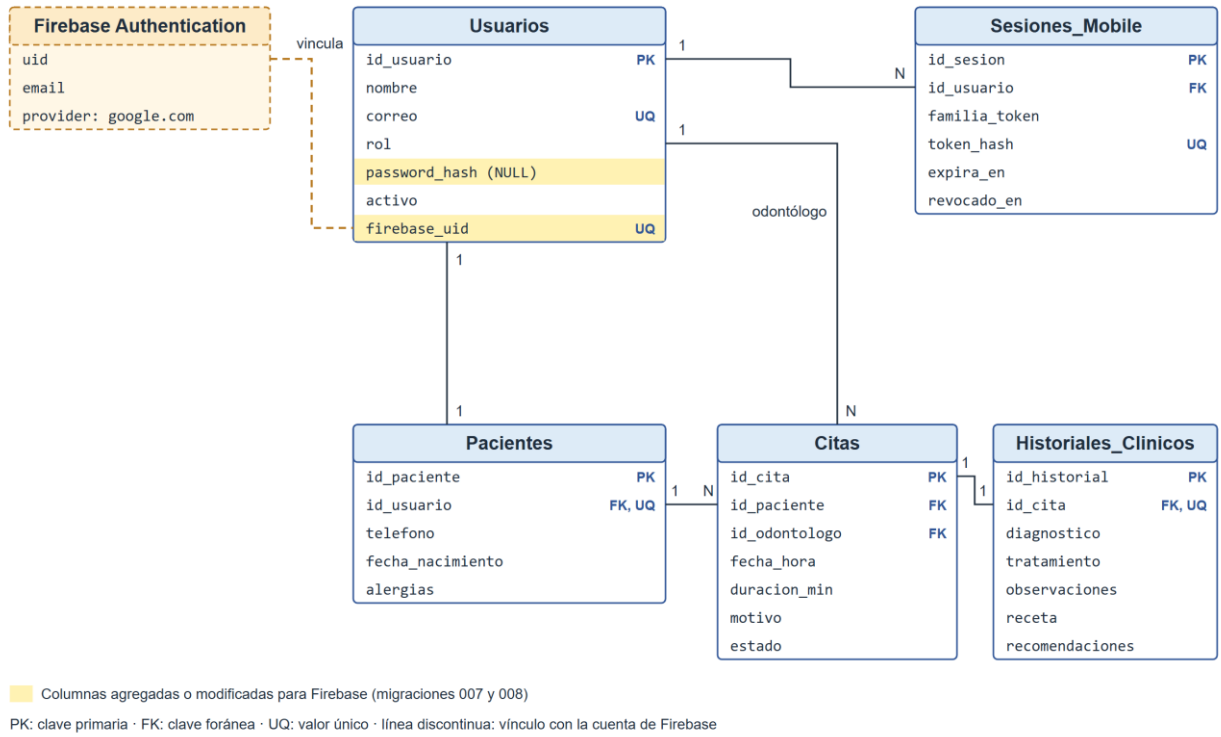
La relación entre ambas partes se hace con la columna `firebase_uid` de la tabla Usuarios. A partir de ahí, cada paciente está ligado a su usuario, y sus citas e historiales clínicos están ligados al paciente. Cuando la aplicación hace una solicitud, envía el token de acceso, que identifica al usuario, y los servicios del backend filtran la información con ese identificador. Por esa razón, un paciente solo puede ver sus propias citas e historiales, aunque intente modificar los parámetros de la solicitud.

Como la web y la aplicación móvil consultan los mismos registros, una cita que el paciente solicita desde el teléfono aparece de inmediato en la agenda de recepción.

La Figura 5 muestra las tablas que intervienen en esta relación y las columnas que se agregaron o modificaron para la integración con Firebase.

Figura 5

Modelo de datos con la vinculación de la cuenta de Firebase



Nota. Elaboración propia a partir de las migraciones de la base de datos del proyecto (404 DPS Devs, 2026a). Se muestran solo las columnas relevantes para la autenticación y la consulta de registros del paciente.

VII. Seguridad de la información

Varias medidas de seguridad ya se mencionaron en las secciones anteriores. En resumen, la integración se protege de la siguiente forma:

- Ninguna credencial forma parte del repositorio: los archivos de configuración de Firebase están en el `.gitignore` y las claves de Firebase Admin solo existen en el servidor.
- La aplicación nunca decide por sí sola quién es el usuario. El servidor valida el token con Firebase Admin y comprueba que no haya sido revocado.
- Además del token, el servidor exige que el correo esté verificado, que el proveedor sea Google, que la cuenta esté activa y que el rol sea de paciente.

- El índice único de `firebase_uid` impide que una misma cuenta de Google quede asociada a dos usuarios.
- Las sesiones tienen una duración corta y se revocan al cerrar sesión, al desactivar la cuenta o al detectar que un token de renovación se reutilizó.
- Los datos que llegan al servidor, como los del formulario de registro, se validan con Zod antes de procesarse.

VIII. Módulos secundarios e historial de registros

El inicio de sesión con Google se integró sobre los módulos construidos para el avance anterior (404 DPS Devs, 2026b), que funcionan igual sin importar la forma en que el paciente haya entrado. La Tabla 5 resume lo que ofrece cada uno.

Tabla 5
Módulos de la aplicación móvil

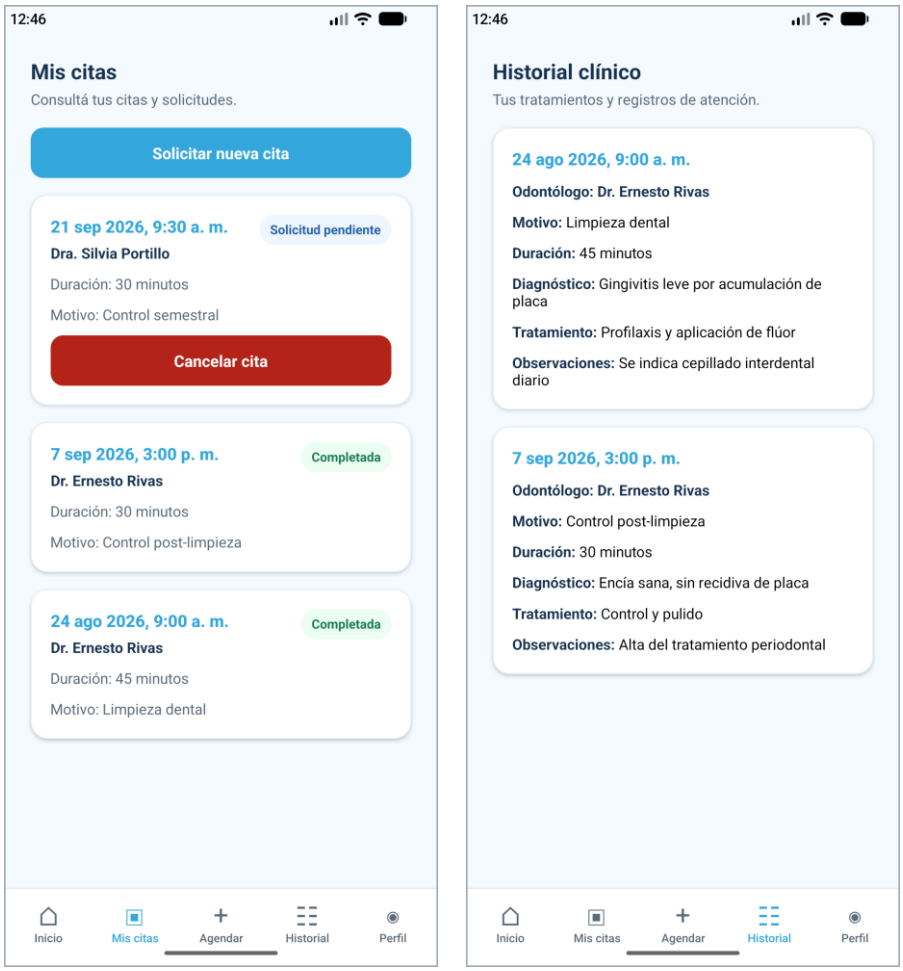
Módulo	Qué permite hacer	Datos que consulta
Inicio	Ver la próxima cita confirmada y cuántas solicitudes siguen pendientes	Citas del paciente
Mis citas	Revisar cada cita con su estado, odontólogo, fecha y motivo, y cancelarla después de confirmar	Citas del paciente
Agendar	Solicitar una cita paso a paso: odontólogo, duración, fecha, horario disponible y motivo	Disponibilidad calculada por el servidor
Historial	Consultar cada atención con diagnóstico, tratamiento, observaciones, receta y recomendaciones	Historiales clínicos del paciente
Perfil	Actualizar el teléfono y las alergias, y cerrar sesión	Usuario y paciente

Nota. Elaboración propia a partir de la aplicación implementada.

Historial y Mis citas son las pantallas donde el paciente consulta los registros guardados en el sistema. Cada persona ve únicamente los suyos, con las fechas en un formato fácil de leer.

Figura 6

Pantallas Mis citas e Historial



Nota. Elaboración propia.

IX. Notificaciones con Firebase Cloud Messaging

9.1. Avance actual

El archivo `mobile/app/(app)/_layout.tsx` comprueba la variable `EXPO_PUBLIC_ENABLE_PUSH`. Si su valor es true y existe un usuario autenticado, solicita permiso, ejecuta `getDevicePushTokenAsync` de `expo-notifications` y registra el token por medio de la API. En `mobile/src/api/notifications.ts` existen funciones para registrar y desregistrar el dispositivo. En el backend, la ruta `/api/v1/notifications/devices` recibe la petición y almacena el token asociado al usuario en la tabla DeviceTokens.

La función está desactivada por defecto y todavía no existe un proceso que consulte las citas y envíe recordatorios a través de FCM. Por lo tanto, lo correcto es hablar de una base preparada para notificaciones, no de una integración terminada.

9.2. Separación de responsabilidades

La propuesta conserva MySQL como fuente principal de datos y el backend Next.js como encargado de las reglas de negocio. Firebase no decide quién puede consultar un historial, confirmar una cita o modificar un perfil. Su función consiste en entregar un mensaje a un dispositivo cuando el servidor determina que existe un aviso válido y oportuno (Firebase, s. f.-d). La Tabla 6 describe el flujo propuesto.

Tabla 6
Flujo técnico propuesto

Paso	Componente	Responsabilidad
1	App React Native	Solicitar autorización para recibir avisos; obtener un token nativo.
2	API Next.js	Validar sesión y vincular el token con el usuario autenticado.
3	MySQL	Conservar citas, estados y registro de dispositivos.
4	Proceso programado del backend	Detectar citas confirmadas que necesitan un recordatorio.
5	Firebase Cloud Messaging	Entregar el aviso al dispositivo Android habilitado.
6	App React Native	Mostrar el aviso y permitir abrir Mis citas para consultar el detalle.

Nota. Elaboración propia.

9.3. Qué token obtiene la aplicación

La implementación actual invoca `getDevicePushTokenAsync`. Según Expo, ese método devuelve un token nativo: en Android corresponde al mecanismo FCM; en iOS corresponde a Apple Push Notification service (APNs). No es un Expo Push Token. En consecuencia, el servidor no debe enviar ese valor a Expo Push Service como si fuera un token de Expo (Expo, s. f.-c).

Para reducir riesgos, se propone validar primero Android con FCM y documentar por separado la incorporación de iOS. Si más adelante se busca unificar el envío por medio del

servicio de Expo, habrá que utilizar `getExpoPushTokenAsync` y revisar qué token conserva la API.

Fragmento simplificado del flujo móvil ya presente en el proyecto:

```
const permission = await Notifications.requestPermissionsAsync();
if (permission.status !== "granted") return;
const nativeToken = await Notifications.getDevicePushTokenAsync();
await registerDeviceToken(String(nativeToken.data));
```

Este fragmento, por sí solo, no demuestra que el dispositivo ya pueda recibir avisos. La confirmación requiere un token válido, un backend capaz de enviarlo al proveedor correcto y una recepción exitosa en un teléfono instalado.

9.4. Actividades pendientes

Con la integración de Authentication ya quedaron listas varias piezas que FCM también necesita: el proyecto de Firebase, la aplicación Android registrada con su `google-services.json`, la compilación de desarrollo (desde Expo SDK 53, las notificaciones remotas no funcionan en Expo Go) y el SDK de Firebase Admin en el servidor. Lo que falta es lo siguiente:

- Manejar los casos de permiso denegado, registrar los cambios de token y desasociar el dispositivo al cerrar sesión, guardando también el proveedor de mensajería para no confundir tokens FCM, APNs y Expo.
- Crear un servicio de mensajería dentro del módulo de notificaciones del backend, que envíe los mensajes con Firebase Admin y las credenciales del servidor.
- Programar una tarea que envíe un recordatorio 24 horas antes de cada cita confirmada y otro una hora antes, que verifique que la cita sigue confirmada y que marque cada envío para evitar duplicados.
- Manejar la zona horaria de la clínica, los errores temporales del servicio, los tokens inválidos y los dispositivos que han sido desregistrados.

- Definir por separado el flujo para iOS. No se declara soporte iOS completo en esta entrega.

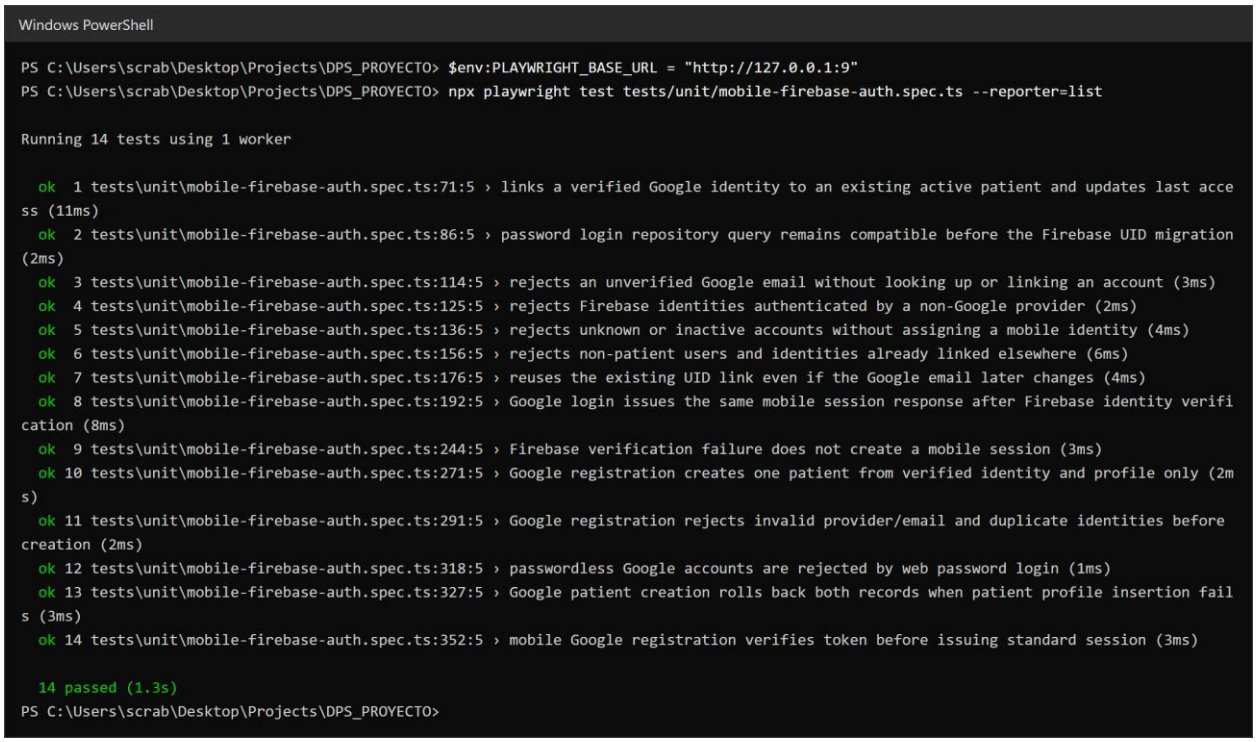
X. Pruebas

Para comprobar la integración, el repositorio cuenta con 14 pruebas unitarias en el archivo `tests/unit/mobile-firebase-auth.spec.ts`. Con ellas se verifica, entre otras cosas, que una cuenta de Google verificada se vincule al paciente correcto; que se rechacen los correos sin verificar, los proveedores distintos de Google y las cuentas inactivas o que no son de pacientes; que no se cree ninguna sesión si falla la verificación del token, y que el registro no deje datos a medias cuando falla una inserción.

Las pruebas se ejecutan con Playwright mediante el comando `npx playwright test tests/unit/mobile-firebase-auth.spec.ts`. Como no necesitan el servidor web, antes de ejecutarlas se define la variable `PLAYWRIGHT_BASE_URL`, que evita que Playwright intente levantarlo. En la Figura 7 se observa que las 14 pruebas pasaron.

Figura 7

Resultado de las pruebas unitarias de la autenticación con Firebase



Nota. Salida de la ejecución de las pruebas en Windows PowerShell, realizada el 4 de noviembre de 2026 sobre la rama main del repositorio (404 DPS Devs, 2026a).

Las pruebas manuales en el emulador de Android se resumen en la Tabla 7.

Tabla 7

Pruebas manuales en el emulador de Android

Caso	Resultado esperado	Resultado obtenido
Inicio de sesión con Google de un paciente ya registrado	Abre las pestañas del paciente con sus datos	**Cumple.** Se abrió Inicio con los datos del paciente.
Registro nuevo con Google	Crea la cuenta de paciente e inicia sesión	**Cumple.** Se creó la cuenta y la aplicación abrió con la sesión iniciada.
Correo de Google sin cuenta en la clínica	Muestra el mensaje de cuenta no encontrada	**Cumple.** Se mostró «No encontramos una cuenta clínica activa con este correo.»
Cancelación del selector de cuentas	Regresa a la pantalla de acceso sin error	**Cumple.** La aplicación volvió a la pantalla de acceso sin mostrar error.
Cierre de sesión e inicio con otra cuenta	No muestra datos del paciente anterior	**Cumple.** La nueva sesión solo mostró los datos de la cuenta que inició.

Caso	Resultado esperado	Resultado obtenido
Inicio de sesión con correo y contraseña	Abre las pestañas del paciente	**Cumple.** Se abrió Inicio con los datos de la paciente.

Nota. Elaboración propia.

XI. Análisis y limitaciones

Para el equipo, la principal ventaja de Firebase fue delegar la verificación de las cuentas de Google: la aplicación nunca maneja contraseñas de terceros y el correo llega ya verificado. Además, el cambio en el backend fue pequeño en comparación con lo que se obtuvo, ya que se limitó a una columna nueva, un servicio que vincula la identidad y la verificación del token. Los permisos y los datos clínicos no tuvieron que modificarse.

La parte más exigente fue pasar de Expo Go a una compilación de desarrollo. Cada prueba requiere compilar e instalar la aplicación, tener el entorno de Android configurado y registrar la huella SHA-1 del certificado, lo que hace más lento el ciclo de trabajo. Aun así, es lo que permite usar el selector de cuentas nativo, que resulta más cómodo para el paciente.

También hay que tener presente que ahora existen dos formas de entrar, con contraseña y con Google, y que eso aumenta los casos que se deben probar; el riesgo se reduce porque ambas terminan en el mismo tipo de sesión. Quedan como limitaciones que la aplicación no recibe cambios en tiempo real ni funciona sin conexión, que la configuración y las pruebas en iOS siguen pendientes, que el acceso con Google es solo para pacientes y que los recordatorios con FCM todavía no están implementados.

XII. Conclusiones

Con esta entrega, la aplicación móvil de Sonrisa Digital permite que los pacientes se registren e inicien sesión con su cuenta de Google. La integración se hizo sin cambiar la manera en que el sistema guarda su información: los datos siguen en MySQL, y la web y el teléfono continúan trabajando sobre los mismos registros.

La decisión más importante fue separar la identidad de los permisos. Firebase confirma quién es el usuario y el backend decide qué puede hacer, revisando el token, el rol y el estado de la cuenta antes de emitir su propia sesión. Gracias a eso, un paciente que entra con Google tiene las mismas garantías que uno que usa contraseña, y ninguna credencial privada quedó en el repositorio.

Para las próximas semanas, el equipo tiene pendiente completar los recordatorios de citas con FCM, aprovechando que la configuración de Firebase ya existe, y probar la aplicación en dispositivos iOS.

XIII. Referencias

404 DPS Devs. (2026a). *Sonrisa Digital* [Software]. GitHub. https://github.com/MiltonMV2025/DPS_PROYECTO

404 DPS Devs. (2026b). *Implementación final (móvil): React Native y Figma en Sonrisa Digital* [Documento no publicado]. Universidad Don Bosco.

Expo. (s. f.-a). *Using Firebase*. Expo Documentation. Recuperado el 3 de noviembre de 2026, de https://docs.expo.dev/guides/using-firebase/

Expo. (s. f.-b). *Introduction to development builds*. Expo Documentation. Recuperado el 3 de noviembre de 2026, de https://docs.expo.dev/develop/development-builds/introduction/

Expo. (s. f.-c). *Notifications*. Expo Documentation. Recuperado el 3 de noviembre de 2026, de https://docs.expo.dev/versions/latest/sdk/notifications/

Expo. (s. f.-d). *SecureStore*. Expo Documentation. Recuperado el 3 de noviembre de 2026, de https://docs.expo.dev/versions/latest/sdk/securestore/

Expo. (s. f.-e). *app.json / app.config.js*. Expo Documentation. Recuperado el 3 de noviembre de 2026, de https://docs.expo.dev/versions/latest/config/app/

Firebase. (s. f.-a). *Firebase Authentication*. Google. Recuperado el 3 de noviembre de 2026, de https://firebase.google.com/docs/auth

Firebase. (s. f.-b). *Verify ID tokens*. Google. Recuperado el 3 de noviembre de 2026, de https://firebase.google.com/docs/auth/admin/verify-id-tokens

Firebase. (s. f.-c). *Add the Firebase Admin SDK to your server*. Google. Recuperado el 3 de noviembre de 2026, de https://firebase.google.com/docs/admin/setup

Firebase. (s. f.-d). *Firebase Cloud Messaging*. Google. Recuperado el 3 de noviembre de 2026, de https://firebase.google.com/docs/cloud-messaging

Invertase. (s. f.-a). *React Native Firebase*. Recuperado el 3 de noviembre de 2026, de https://rnfirebase.io/

Invertase. (s. f.-b). *Social authentication*. React Native Firebase. Recuperado el 3 de noviembre de 2026, de https://rnfirebase.io/auth/social-auth

Román Gallardo, A., Flores Cortés, C. A., y Herrera Morales, J. R. (2020). *Programación de aplicaciones móviles con React Native*. Editorial BUK. https://www.buk.com.mx/9786079878214/description